

CORRIGÉ DU DEVOIR À LA MAISON 11

Résolution d'une équation différentielle d'ordre 2 : oscillateur linéaire

Étude d'un oscillateur harmonique amorti : gestion du recul d'un canon (cf. TD MI2 – exercice 3)

1. Système : canon de masse M assimilé à son centre de gravité G

➤ Référentiel : terrestre supposé galiléen, base cartésienne

➤ Bilan des forces

- Poids : $\vec{P}$: force conservative dérivant de $E_{P,pes} = cste$

- Réaction normale du sol : $\vec{R}_N$ force non conservative telle que $W(\vec{R}_N) = 0$

- Force de rappel élastique :

$$\vec{F} = -k_2(l - L_0)\vec{u}_{\text{sortant}} = -k_2(OG - L_0)\vec{u}_x = -k_2x\vec{u}_x : \text{force conservative}$$

dérivant de $E_{P,elas} = \frac{1}{2}k_2x^2$

- Force de frottements fluides visqueux : $\vec{F}_f = -\lambda\vec{v} = -\lambda\dot{x}\vec{u}_x$: force non conservative

➤ Système non conservatif à un degré de liberté : x

➤ Énergie cinétique : $E_C = \frac{1}{2}M\dot{x}^2$

➤ Énergie mécanique : $E_m = E_C + E_{P,elas} + E_{P,pes} = \frac{1}{2}M\dot{x}^2 + \frac{1}{2}k_2x^2 + cste$

➤ Théorème de la puissance mécanique :

$$\frac{dE_m}{dt} = \mathcal{P}^{NC} = \mathcal{P}(\vec{R}_N) + \mathcal{P}(\vec{F}_f) = -\lambda\vec{v} \cdot \vec{v} = -\lambda\dot{x}^2$$

$$\frac{d}{dt}\left(\frac{1}{2}M\dot{x}^2\right) + \frac{d}{dt}\left(\frac{1}{2}k_2x^2\right) = -\lambda\dot{x}^2 \Leftrightarrow \frac{d}{dx}\left(\frac{1}{2}M\dot{x}^2\right)\frac{dx}{dt} + \frac{d}{dx}\left(\frac{1}{2}k_2x^2\right)\frac{dx}{dt} = -\lambda\dot{x}^2$$

$$M\ddot{x} + k_2x\dot{x} = -\lambda\dot{x}^2 \Leftrightarrow M\ddot{x} + k_2x = -\lambda\dot{x} \Leftrightarrow \ddot{x} + \frac{\lambda}{M}\dot{x} + \frac{k_2}{M}x = 0$$

$$\boxed{\frac{d^2x(t)}{dt^2} + 2\xi\omega_0\frac{dx(t)}{dt} + \omega_0^2x(t) = 0}$$

avec $\omega_0 = \sqrt{\frac{k_2}{M}}$ et $2\xi\omega_0 = \frac{\lambda}{M}$ soit $\xi = \frac{\lambda}{2M\omega_0} \Leftrightarrow \xi = \frac{\lambda}{2\sqrt{Mk_2}}$

Résolution numérique de l'équation différentielle avec Python

```

8  ## Cellule 1 : Importation des bibliothèques utiles
9  import numpy as np
10 import matplotlib.pyplot as plt
11 import scipy.integrate as sci    # Pour utiliser la fonction odeint
12 from math import *
```

```

14 ## Cellule 2 : Données du problème physique -> A COMPLETER
15
16 # Constantes physiques
17 ksi = 0.1 # Coefficient d'amortissement
18 M = 800. # Masse du canon (kg)
19 m = 2. # Masse de l'obus (kg)
20 k2 = 244. # Constante de raideur du ressort (N/m)
21 v0 = 600. # Vitesse initiale de l'obus (m/s)
22 omega = sqrt(k2 / M) # Expression de la pulsation propre (en rad.s-1)
23 T = 2*np.pi/omega # Expression de la période propre (en s)
24
25 # Paramètres de la résolution numérique
26 t0, tf = 0, 5*T # bornes de l'intervalle de résolution
27 n = 200 # nombre de points
28 dt = (tf-t0) / (n-1) # expression du pas temporel pour la résolution numérique
29
30 # Conditions initiales
31 x0 = 0 # Abscisse initiale du canon (m)
32 xprim0 = - m / M *v0 # Expression de la vitesse initiale du canon (m/s)
33 y0 = np.array([x0, xprim0]) # Initialisation du vecteur y=[x,x']
34
35 # Création de la variable temporelle
36 """
37 numpy.linspace (tmin,tmax,N)
38 Renvoie un tableau de N points régulièrement espacés
39 entre tmin (inclus) et tmax (inclus)
40 """
41 t = np.linspace(t0,tf,n) # n points régulièrement espacés entre t0 (inclus) et tf (inclus)
42
43 # Définition de la fonction dérivée de y(t)
44 def derivee_y(y,t):
45     return np.array([y[1], -2*ksi*omega*y[1] - omega**2*y[0]]) # expression à compléter

```

```

47 ## Cellule 3 : Résolution avec odeint et représentation graphique -> A COMPLETER
48 """
49 scipy.integrate.odeint (f,y0,t)
50
51 Résout un système d'équations différentielles d'ordre 1
52 Paramètres :
53     f : f(y,t) : fonction qui calcule la dérivée de y à l'instant t
54     y0 : tableau ou vecteur : condition initiale
55     t : tableau d'instants pour lesquels la résolution est réalisée
56 Renvoie :
57     y : tableau ou vecteur avec les valeurs de y calculées pour chaque instant t
58     (les valeurs initiales sont sur la 1ère ligne)
59 """
60 y = sci.odeint(derivee_y,y0,t) # Vecteur y obtenu avec odeint : à compléter
61
62 # Représentation graphique
63 plt.subplot(2,1,1) # Subdivision de la fenêtre graphique en 2 fenêtres, l'une au-dessus de l'autre
64 # et tracé sur la fenêtre du haut
65 plt.plot(t,y[:,0],'r-',label='odeint') # Graphe de l'abscisse x en fonction du temps en trait
66 # rouge : à compléter
67 plt.xlabel(r"$t$ (en s)") # Nom de l'axe des abscisses
68 plt.ylabel("Position (m)") # Nom de l'axe des ordonnées
69 plt.legend(loc = 'upper right') # Affichage de la légende en haut à droite
70 plt.grid() # Affichage de la grille
71 plt.show() # Affichage de la fenêtre
72
73 plt.subplot(2,1,2) # Tracé sur la fenêtre du bas
74 plt.plot(t,y[:,1],'r-',label='odeint') # Graphe de la vitesse xprim en fonction du temps en trait
75 # rouge : à compléter
76 plt.xlabel(r"$t$ (en s)") # Nom de l'axe des abscisses
77 plt.ylabel("Vitesse (m/s)") # Nom de l'axe des ordonnées
78 plt.legend(loc = 'upper right') # Affichage de la légende en haut à droite
79 plt.grid() # Affichage de la grille
80 plt.show() # Affichage de la fenêtre

```

```

79 ## Cellule 4 : Résolution avec Euler et représentation graphique -> A COMPLETER
80 def euler(F, y0, t, dt, n):
81     """
82     Paramètres :
83         F : fonction donnant y'
84         y0 : condition initiale sur y
85         t : tableaux des instants pour lesquels les calculs sont réalisés
86         dt : pas de discrétisation utilisé pour la résolution
87         n : nombre de points pour lesquels les calculs sont réalisés
88     Renvoie :
89         y : vecteur contenant l'ensemble des valeurs approchées yk
90     """
91     y = np.zeros([n,2]) # initialisation du vecteur y : n lignes, 2 colonnes
92     y[0] = y0 # prise en compte des conditions initiales

```

```

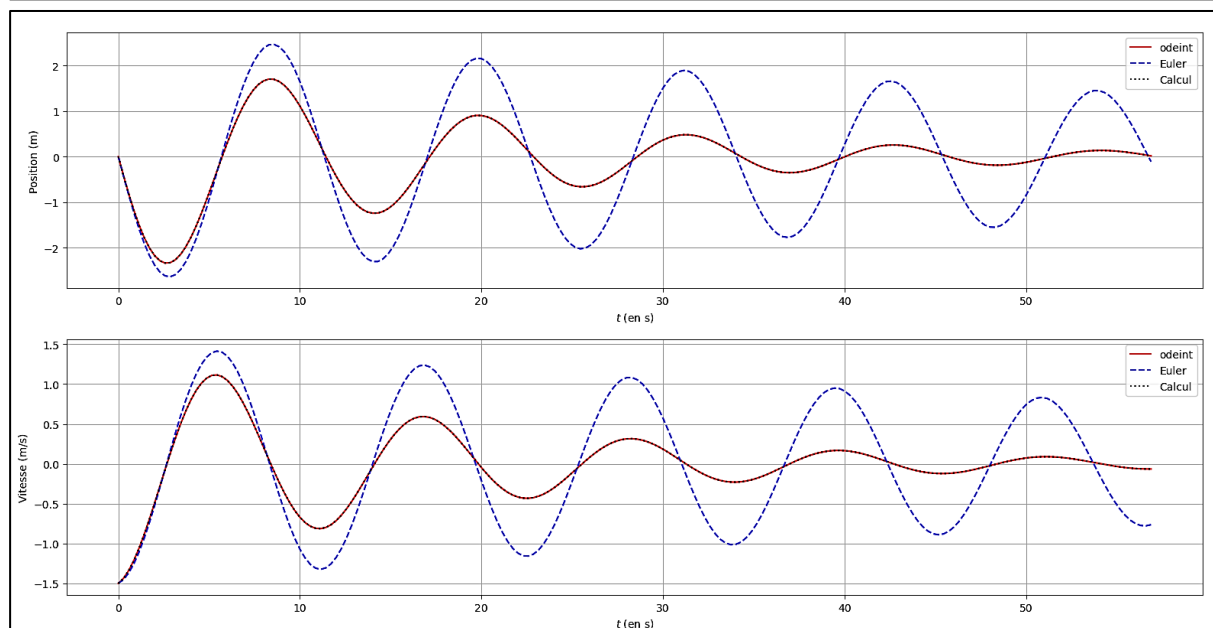
93
94     # Boucle permettant le calcul des yk par récurrence
95     for k in range(0,n-1):         # k prend les valeurs de 0 à n-2
96         y[k+1] = y[k] + F(y[k],t[k])*dt # relation de récurrence : à compléter
97     return y
98
99 y_euler = euler(derivee_y, y0, t, dt, n) # Vecteur y_euler obtenu avec la méthode d'Euler : à
compléter
100
101 # Représentation graphique
102 plt.subplot(2,1,1)
103 plt.plot(t,y_euler[:,0],'b--',label='Euler') # Graphe de l'abscisse x en fonction du temps en
tirets bleus
104 plt.legend(loc = 'upper right')
105 plt.subplot(2,1,2)
106 plt.plot(t,y_euler[:,1],'b--',label='Euler') # Graphe de la vitesse xprim en fonction du temps en
tirets bleus
107 plt.legend(loc = 'upper right')

```

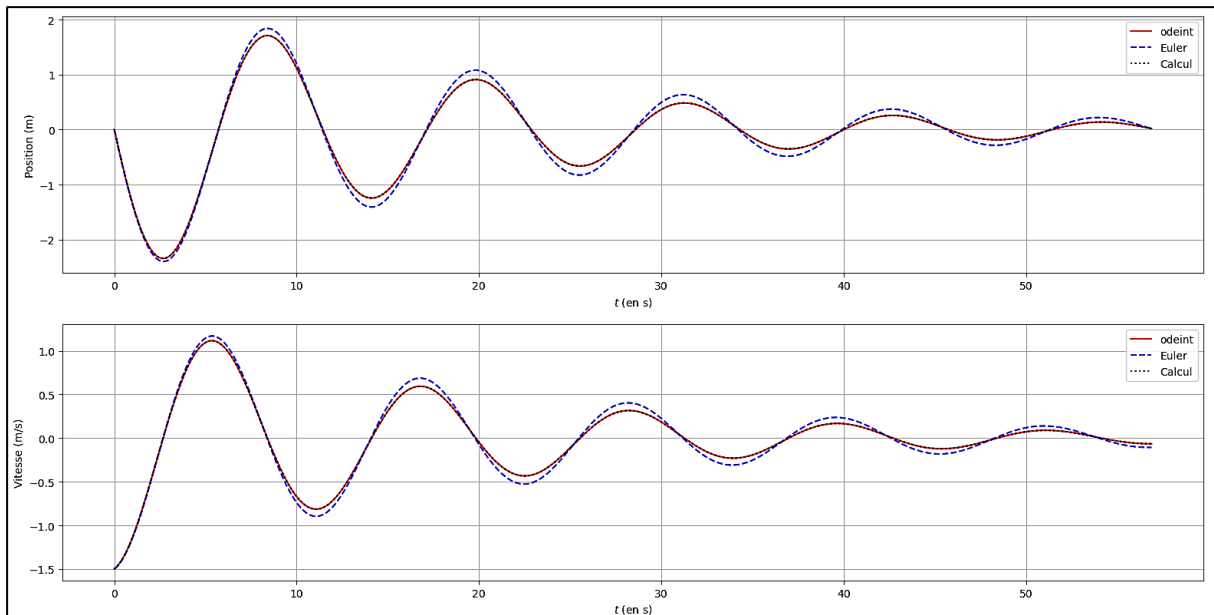
```

109 ## Cellule 5 : Calcul analytique et représentation graphique -> A COMPLETER
110 wp = omega*sqrt(1-ksi**2) # Expression de la pseudo-pulsation (rad/s)
111 x_calc = xprim0 / wp * np.sin(wp*t) * np.exp(-ksi*omega*t) # Expression analytique de l'abscisse
112 xprim_calc = xprim0 *(np.cos(wp*t) - ksi*omega / wp * np.sin(wp*t)) * np.exp(-ksi*omega*t) #
Expression analytique de la vitesse
113
114 # Représentation graphique
115 plt.subplot(2,1,1)
116 plt.plot(t,x_calc,'k:',label='Calcul') # Graphe de l'abscisse x en fonction du temps en pointillés
noirs : à compléter
117 plt.legend(loc = 'upper right')
118 plt.subplot(2,1,2)
119 plt.plot(t,xprim_calc,'k:',label='Calcul') # Graphe de la vitesse xprim en fonction du temps en
pointillés noirs : à compléter
120 plt.legend(loc = 'upper right')

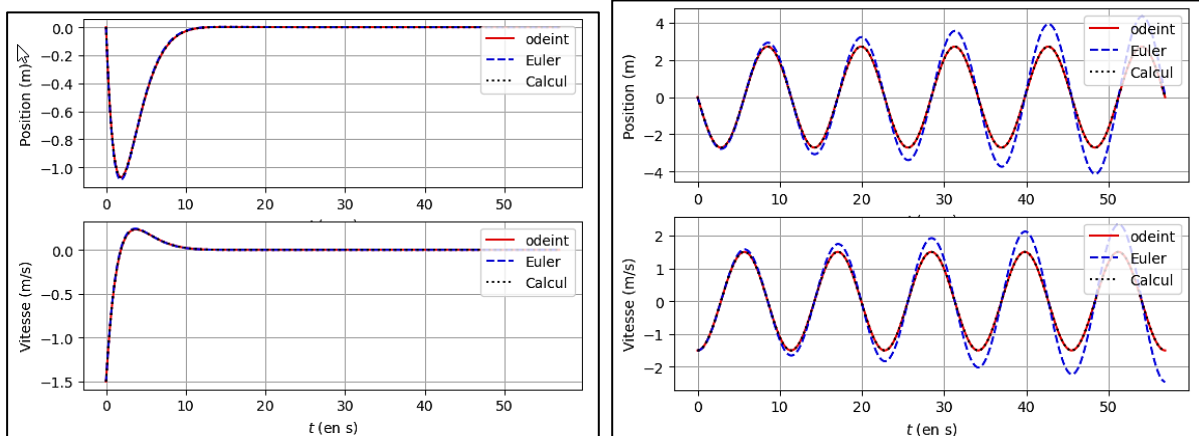
```



- Commentaires : Les graphes obtenus avec la méthode d'Euler diffèrent de ceux obtenus avec la fonction `odeint`, qui sont similaires à ceux obtenus avec le calcul analytique, ce qui montre la performance de l'algorithme de Runge-Kutta.
- Distance de recul $d_m = 2,3 \text{ m}$ à l'instant $t_m = 2,6 \text{ s}$
- ❖ **Influence du nombre de points**
- Commentaires : Pour $n = 1000$ points, les performances de la méthode d'Euler explicite s'améliorent : les courbes se rapprochent de celles obtenues analytiquement. L'augmentation du nombre de points et donc la diminution du pas de résolution permettent de réduire l'erreur commise.



❖ Influence du coefficient d'amortissement



➤ Commentaires : Pour $n = 1000$ points et $\xi = 0,9$ (figure de gauche) : la méthode d'Euler donne des résultats convenables. Pour $\xi = 0$ (figure de droite), la solution obtenue avec la méthode d'Euler diverge ! Dans tous les cas, l'algorithme de Runge-Kutta à pas variable de la fonction odeint fournit une solution numérique identique à la solution analytique.

➤ Dans le cas du régime critique, i.e. pour $\xi = 1$, la méthode d'Euler fournit une solution très proche de celle fournie par l'algorithme de Runge-Kutta.

La distance de recul est $d'_m = 1,0 \text{ m}$ à l'instant $t'_m = 1,8 \text{ s}$: on retrouve les valeurs calculées en TD.

